

Agency Manager — Developer Guide

Version 1.4.0 — architecture, hooks, and extension points

1. Architecture

Namespace: AgencyManager\. A small PSR-4-ish autoloader (includes/autoload.php) maps namespace segments to hyphenated file names — WordPress file-naming convention combined with real PHP namespaces.

Every subsystem class exposes a register() method that wires its own hooks; includes/class-plugin.php is the single place that instantiates all of them. No subsystem reaches into another's internals directly.

Rendering is centralized, not duplicated: Frontend\Carousel_Renderer backs every grid/carousel/slider/featured shortcode AND every matching Elementor widget AND the admin Preview tab. Frontend\Card_Renderer renders one card; Carousel_Renderer wraps N of them. There is exactly one template per card type.

No raw SQL anywhere — only WP_Query / get_posts / wp_insert_post / get_terms / wp_insert_term and equivalent core APIs.

2. Registration Guard & Theme Compatibility

Compat\Registration_Guard decides whether Agency Manager registers the talent/location post types, their taxonomies, and the profile meta boxes, or defers to something that already provides them (e.g. a theme with its own Talent/Location system).

- CPT/taxonomy registration: generic post_type_exists()/taxonomy_exists() checks, run on the init hook at priority 20 (not at plugin-construction time, since a theme's functions.php hasn't executed yet when the plugin file first loads).
- Meta boxes: the same generic check, plus one narrow defined('EDEN_CAST_DIR') check — isolated to this one class so the rest of the plugin stays fully theme-agnostic.
- Forms (am_form/am_submission), the Dashboard, Applications, Settings, Import/Export, shortcodes, and Elementor widgets are never guarded — nothing else on a typical site registers anything equivalent.

3. Data Model

What	Where
Post meta	_am_* prefix, e.g. _am_age, _am_city, _am_gallery_ids, _am_featured, _am_homepage, _am_active, _am_social_instagram/_facebook/_tiktok/_website
Term meta	am_* prefix (am_group_image_id, am_group_button_text, am_group_button_url) on talent_group and location_type

What	Where
Settings	One autoloaded option, <code>am_settings</code> ; request-memoized via <code>Settings::all()</code>
Gallery	<code>_am_gallery_ids</code> is a comma-separated attachment-ID string; parsed defensively wherever read
Scouting Images	<code>placeholder.{type}.image_ids</code> is a plain array of attachment IDs in <code>am_settings</code> . <code>Card_Renderer::render_placeholder_cards()</code> cycles through it by card index (<code>\$images[\$i % count(\$images)]</code>); empty array falls back to no image (plain placeholder block)
<code>location_type</code>	Taxonomy key is unchanged; only its admin label reads "Categories" — no data migration involved

4. Template Overrides

A theme can override any of the four card/placeholder templates without touching the plugin: create `{theme}/agency-manager/{template-name}.php` — checked before the plugin's own `templates/{template-name}.php`.

```
your-theme/
```

```
agency-manager/
```

```
talent-card.php <- checked first
```

Overridable names: `talent-card`, `location-card`, `talent-placeholder-card`, `location-placeholder-card`. Copy the plugin's version as a starting point; each receives the same variables the plugin passes (`$post_id` for real cards, `$config` for placeholder cards). This is also exactly what the admin Preview tab renders, so an override applies there too — one template, one visual result everywhere.

Archive/single templates

Frontend\Template_Loader hooks `template_include` (only for `is_singular/is_post_type_archive` on `talent/location`) and resolves via `Templates::locate_page_template()`, a 3-tier lookup: (1) `{theme}/agency-manager/{name}.php`, (2) `locate_template([name].php)` — the theme's own conventionally-named root template, WordPress's native template-hierarchy naming, (3) the plugin's own `templates/{name}.php` as a last resort. Names: `single-talent`, `single-location`, `archive-talent`, `archive-location`.

Because tier 2 defers to a theme's own native file, a theme that already registers the CPTs and ships its own `single-talent.php` (e.g. Eden Cast) sees zero behaviour change — `Template_Loader` only matters on a standalone install with no theme-provided template at all.

4b. Theme Style

Agency Manager has no style system of its own beyond a fixed set of CSS custom properties, each with a neutral fallback: `--am-primary`, `--am-secondary`, `--am-accent`, `--am-text`, `--am-heading`, `--am-button`, `--am-border`, `--am-radius`. There is no PHP-side preset registry, no admin setting, no shortcode attribute, and no Elementor control for appearance — `assets/css/frontend.css` consumes these variables via

`var(--am-x, <neutral-fallback>)` throughout, and that is the entire mechanism.

A theme defines its own look by declaring these variables in its own stylesheet (a plain `:root{--am-primary:#...;}` block), and by targeting Agency Manager's plain, stable class names directly (`.am-talent-card`, `.am-talent-card__meta`, `.am-scouting-card__badge`, etc.) for typography/spacing/hover motion that isn't a colour variable. This is exactly how `wp-content/themes/eden-cast/assets/css/talent-locations.css` implements Eden Cast's own maroon/gold/serif identity — entirely in theme code, with a 'body' selector prefix to guarantee it wins regardless of stylesheet enqueue order, and zero Eden Cast branding anywhere in the plugin.

This intentionally makes it a theme's job, not the plugin's, to decide what a Talent/Location card looks like beyond the neutral default — matching the stated architecture: the plugin controls data, rendering, widgets, shortcodes, import/export, and applications; the theme controls colours, typography, spacing, and templates.

4c. Meta Compatibility (Meta_Resolver)

`Frontend\Meta_Resolver::get($post_id, $type, $field)` is the read path every card/single template now uses instead of a raw `get_post_meta($post_id, '_am_'. $field, true)` call. It reads `_am_{$field}` first; if that's empty, it checks the `am_meta_fallback_map` filter for a fallback meta key and reads that instead.

Agency Manager ships zero entries in this map and contains no reference to any specific theme or plugin. A theme that already owns Talent/Location content under its own meta prefix hooks `am_meta_fallback_map` once, in its own code, to make that existing content display fully through Agency Manager's rendering surfaces (cards, single templates) without migrating or duplicating a single value.

```
add_filter( 'am_meta_fallback_map', function ( $map, $type, $post_id ) {  
    if ( 'talent' === $type ) {  
        $map['city'] = '_ec_city';  
        $map['age'] = '_ec_age';  
    }  
    return $map;  
}, 10, 3 );
```

Because the `_am_*` value always wins when present, this is strictly additive: a standalone install where Agency Manager owns its own Talent/Location records (real `_am_*` data written by its own tabbed editor or the Applications publish flow) behaves identically whether or not anything hooks the filter.

5. Hooks (Actions & Filters)

Hook	Type	Fires / Signature
<code>am_submission_created</code>	action	A new form submission is stored. (int <code>\$submission_id</code>)

Hook	Type	Fires / Signature
am_submission_status_changed	action	Any workflow status change. (int \$submission_id, string \$new_status, string \$old_status)
am_submission_published	action	publish_submission() creates the real post. (int \$post_id, int \$submission_id)
am_before_import_section	action	Before an import section is processed. (string \$section, mixed \$payload)
am_after_import_section	action	After a section finishes. (string \$section, array \$counts)
am_form_fields	filter	A form's field schema. (array \$fields, int \$form_id)
am_notification_recipient	filter	Recipient of the new-submission email. (string \$to, int \$submission_id)
am_settings_defaults	filter	The plugin's default settings structure. (array \$defaults)
am_talent_query_args	filter	Final WP_Query args for any talent listing.
am_location_query_args	filter	Final WP_Query args for any location listing.
am_meta_fallback_map	filter	An _am_* field read is empty. (array \$map, string \$type, int \$post_id) -> field=>meta_key entries

Example

```
add_filter( 'am_notification_recipient', function ( $to, $submission_id ) {
    return $to . ',bookings@example.com';
}, 10, 2 );
```

6. Folder Structure

agency-manager/

agency-manager.php Plugin header, constants, autoloader

uninstall.php Removes only the am_settings option

includes/

class-plugin.php Bootstrap - wires every subsystem onto its hook

class-activator.php Seeds default forms/settings, wizard redirect

class-settings.php The am_settings option: read/write/defaults

compat/class-registration-guard.php

cpt/ Talent/Location CPTs, taxonomies, tabbed

meta boxes, term meta (all guarded)

forms/ am_form/am_submission CPTs, Form_Renderer,

Workflow, Notifications, Csv_Exporter

```

frontend/ Templates, Template_Loader (archive/single),
Card_Renderer, Carousel_Renderer, Query,
Meta_Resolver (am_meta_fallback_map)
admin/ Dashboard, Applications, Forms,
Website Display, Import/Export, Settings,
Backup, Shortcode_Reference, Media_Picker_Assets
wizard/ Setup_Wizard
shortcodes/ 8 grid/featured/carousel/slider shortcodes
+ 2 scouting shortcodes
elementor/ Elementor_Integration, Widget_Style_Presets,
widgets/
export-import/ Schema, Exporter, Importer
templates/ Default card/placeholder templates
assets/
css/ frontend.css, carousel.css
js/ carousel.js
admin/ admin.css, admin.js, tabs.js,
shortcode-reference.js (copy/search/builder)
elementor/ reset-style-defaults.js, widget-style-presets.js
(both editor-only)

```

7. Extending the Plugin

Adding a new field to Talent or Location

- Add the field to the relevant tab callback in Cpt\Meta_Boxes (render_talent_box() or render_location_box()), reusing render_field_row() or render_media_picker_row().
- Add the meta key to the save loop in save_talent()/save_location() with the appropriate sanitizer.
- If it should appear on the frontend card, add it to the relevant template in templates/.

Adding a new shortcode/widget filter parameter

- Extend Frontend\Query::build_args() to accept and translate the new parameter into WP_Query args.
- Add the corresponding shortcode_atts() entry in Shortcodes\Shortcodes.
- Add the corresponding Elementor control in Elementor\Widgets\Base_Grid_Widget and read it in render().

Adding an Elementor widget

Extend `Base_Grid_Widget` (for a listing) or `Base_Form_Widget` (for a form) and register it in `Elementor\Elementor_Integration` on the `elementor/widgets/register` hook — never `elementor/loaded` directly, and never reference an Elementor class outside a hook that only fires when Elementor is active.

Adding an Elementor Style-tab control

`Base_Grid_Widget::register_style_controls()` holds all six Style-tab sections (`Image/Card/Button/Badge/Typography/Spacing`). Add a new `add_control()/add_responsive_control()` call there with a 'selectors' (or 'selector' for a `Group_Control_*`) entry targeting the relevant `.am-*` class — no `render()/PHP` change is needed; Elementor compiles the CSS itself, scoped to `{{WRAPPER}}` (that one widget instance's unique ID). This is why none of these controls appear in `Carousel_Renderer` or `Card_Renderer` at all.

Adding a shortcode to the Shortcode Reference panel

Add an entry to the relevant group's 'shortcodes' array in `Admin\Shortcode_Reference::groups()` — tag, example, description, when (the 'When to use it' text), params. It immediately appears on the Dashboard and any admin screen that calls `render_panel()` with that group key, including the interactive builder (if params is non-empty) and the search index — no other wiring is needed.

Adding a parameter to the Shortcode Builder

Add an entry to `Admin\Shortcode_Reference::PARAM_CONTROLS` keyed by the real shortcode attribute name (type: text/number/select/checkbox, label, and options for select) — `assets/admin/shortcode-reference.js`'s `buildShortcode()` reads any element with a matching `data-param` attribute generically, so no JS change is needed for a new parameter, only the PHP control definition and adding the param name to the relevant shortcode's 'params' array.

How "Reset to Theme Defaults" works

`Base_Grid_Widget` registers a `Controls_Manager::BUTTON` control (name: `reset_style_defaults`) with 'event' => `'am:reset_style_defaults'`, inside its own `'style_reset_section'` at the top of the Style tab. Elementor fires that exact event string on `elementor.channels.editor` when the button is clicked. `assets/elementor/reset-style-defaults.js` (enqueued editor-only via `elementor/editor/after_enqueue_scripts` in `Elementor_Integration::enqueue_editor_scripts()`) listens for it, reads `elementor.config.widgets[widgetType].controls` to find every control whose 'tab' is 'style' (excluding its own `META_CONTROLS` exclusion list — the `Reset/Widget-Style-preset` controls themselves, which also live in Style-tab sections but aren't style values), and calls `model.setSetting()` with each one's registered 'default' — writing only to the currently-edited element's own Backbone settings model, so no other widget instance, page, or the theme's CSS is ever touched. No PHP-side reset logic exists; it's entirely a client-side editor interaction, matching the fact that all Style-tab controls are themselves pure client-side Elementor 'selectors' definitions with no PHP `render()` counterpart.

How Widget Style Presets work

Elementor\Widget_Style_Presets stores presets as one slice of `am_settings` (`Settings::get_widget_style_presets()`, `key` = preset name, `value` = map of control name => value). Its three `wp_ajax_am_*_widget_style_preset` handlers (`save/delete/rename`, all `manage_options`-gated and nonce-verified against `Widget_Style_Presets::nonce_action()`) are the only server-side logic — Load never calls the server at all.

`Elementor_Integration::enqueue_editor_scripts()` localizes the full preset list (`amWidgetStylePresets.presets`, names and values both) to `assets/elementor/widget-style-presets.js` once per editor page load. Save reads every current Style-tab control's value via `model.getSetting()` (same enumeration technique as Reset, using its own identical `META_CONTROLS` exclusion list) and POSTs it to the save handler; Load looks the selected preset straight up in the localized presets object and applies it via `model.setSetting()` + `editedElementView.renderOnChange()` — no round trip. After Save/Rename/Delete succeed, the script updates both its local cache and the live `<select data-setting="widget_style_preset_select">` element's options directly via the DOM, so the change is visible without an editor reload.

Import/Export treats `widget_style_presets` exactly like every other named section: `import_settings_slice(['widget_style_presets' => $payload])` relies on `array_replace_recursive()` merging the sub-array key-by-key (preset name = key), which is naturally an upsert-by-name — no custom merge logic was needed.

8. Coding Standards & Compatibility

- WordPress Coding Standards followed throughout: `esc_html()`/`esc_attr()`/`esc_url()` on all output, `wp_verify_nonce()` + `current_user_can()` on every state-changing admin action.
- Tested clean on PHP 7.4 through 8.3 — no deprecated dynamic-property warnings, no implicit nullable-parameter issues.
- No build step required — plain PHP/CSS/JS, no bundler, no Composer dependency at runtime.